Provisional Patent Disclosure Draft

Status: draft for possible self-filed provisional patent application.

Do not publish this draft. Do not mark FDE as "Patent Pending" unless an application is actually filed.

Title

Recursive Source-Pointer Skill Routing and Verification System for Multi-Runtime AI Operations

Inventor Information

Inventor(s): TBD by human.

Owner/assignee: TBD by human.

Field

The invention relates to AI-assisted software operation, multi-agent workflow routing, source-of-truth management, publication containment, and automated verification of operational decision systems.

Background

AI coding agents, chat agents, browser agents, and external review systems often operate with inconsistent context, duplicated prompt instructions, weak publication boundaries, and unverifiable completion claims. Conventional prompt libraries and workflow documents do not reliably route a task through source truth, safety gates, runtime-specific instructions, and completion checks.

This creates failures including:

• acting from stale or inferred context;
• exposing private source material during public release;
• publishing or externally sending before human review;
• losing traceability between a user request, source files, decision, and

completion evidence;
• duplicating long source documents into runtime prompts; and
• producing many skills without machine-verifiable trigger coverage.

Summary

The disclosed system provides a recursive skill-routing architecture for AI operations. A set of thin runtime skills is generated and arranged in three layers:

1. core skills that protect source truth, scope, public boundaries, fact checks, memory, restart, security, and completion;
2. router skills that select an operating lane, runtime, external AI path, workspace, or publication route; and
3. leaf skills that point to concrete source documents or workflow modules.

Each skill is a source-pointer shim rather than a copy of the underlying source document. The system validates that each skill contains trigger wording, Japanese and English user-trigger surfaces, source pointers, runtime coverage, human approval boundaries, and metadata contracts.

Technical Problem

The system solves the technical problem of maintaining consistent, verifiable, and containment-aware AI operations across multiple runtimes while minimizing context duplication and preventing unsafe publication or external-send actions.

Technical Solution

The solution combines:

• recursive skill layers;
• source-pointer-only skill bodies;
• trigger-surface validation;
• runtime coverage validation;
• public/external/hook/settings/auth containment gates;
• completion verification gates;

• generated metadata for runtime discovery;

• an ordered skill index;

• machine-readable guarantee scripts; and

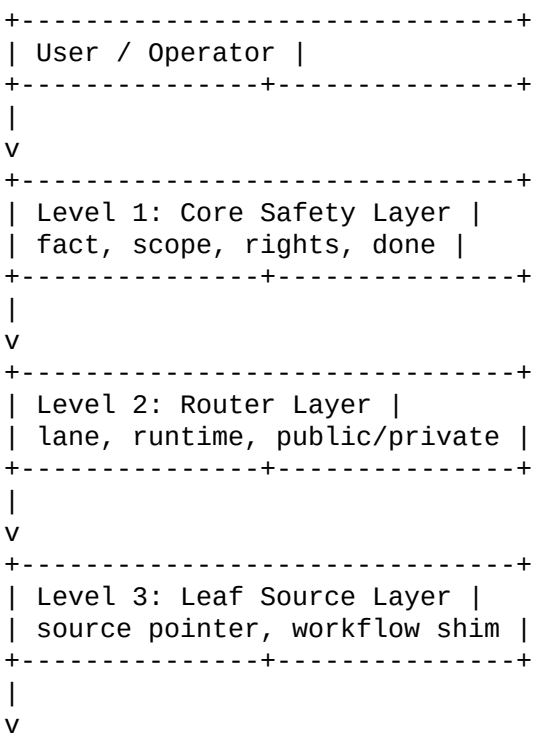
• a publication split between private implementation and public kernel.

Example Architecture

```
User request
-> pre-execution fact check
-> scope routing gate
-> level 1 core skill
-> level 2 router skill
-> level 3 leaf skill
-> source pointer read
-> action or hold
-> done verification closeout
-> publication containment if external/public action is involved
```

Figure Descriptions

Figure 1: Recursive Routing Stack



```
+------------------------------+
| Evidence / Action / Hold |
+------------------------------+
```

Figure 2: Containment and Completion Loop

```
Task packet
-> source truth check
-> risk classification
-> public/external/hook/auth/settings gate
-> execute only if allowed
-> collect evidence
-> done verification
-> final report with fact / inference / unknown separation
```

Figure 3: Public/Private Split

```
Private operating layer
- full recursive skills
- internal source pointers
- validation scripts
- runtime-specific workflows
- absorbed dialogue and internal logs

Reduced public kernel
- concept overview
- abstract recursive map
- four general gates
- rights notice
- restrictive license
```

Example Data Structures

```
SkillSpec:
name: string
level: 1 | 2 | 3
kind: core | router | leaf
source_pointers: list[path_or_uri]
description: trigger text
related_core_skills: list[name]
approval_boundaries: list[action_type]

ValidationResult:
skill_name: string
```

```
frontmatter_ok: boolean
trigger_gap_count: integer
source_pointer_ok: boolean
runtime_coverage_ok: boolean
human_approval_boundary_ok: boolean
metadata_contract_ok: boolean
```

Example Method

1. Receive a user request.
2. Identify whether the request involves current facts, source truth, public release, external send, credentials, hook enablement, settings, auth, memory write, or completion claim.
3. Select one or more core skills based on trigger descriptions.
4. If the task requires routing, select a router skill.
5. If the task requires concrete workflow execution, select a leaf skill.
6. Read the first source pointer listed in the selected skill as the source of truth.
7. Read secondary source pointers only when required by task scope.
8. Execute or hold the task according to the selected skill and source truth.
9. Before public/external action, require human approval in the current conversation.
10. Before completion, verify changed files, command evidence, logs, source pointers, and unverified risk.
11. Emit a compact final report separating facts, inferences, and unknowns.

Example Generator

A generator receives an ordered list of `SkillSpec` records and writes:

- `SKILL.md` for each skill;
- `agents/openai.yaml` metadata for each skill;
- a recursive skill index; and
- validation-ready source pointers and approval boundaries.

Example pseudocode:

```
for each SkillSpec in ordered_skill_specs:
validate name, level, kind, source pointers, and trigger text
create skill directory
write SKILL.md with:
- frontmatter name and trigger description
- source pointer list
- runtime coverage statement
- human approval boundary
- completion evidence requirement
write runtime metadata

write recursive index grouped by importance and level
run trigger-gap validation
run source-pointer validation
run public-action containment validation
```

Example Validation

Validation checks may include:

• every generated skill has `name` and `description` frontmatter;

• each description includes natural trigger wording;

• each description includes Japanese trigger examples;

• no required core skill is missing;

• source pointers are present;

• runtime coverage includes target runtimes;

• public release, external sending, hook enablement, credential changes, settings changes, auth changes, and repository visibility changes require human approval;

• trigger gap count is zero; and

• Brain/FDE source-of-truth contracts are satisfied.

Variations

The system may be applied to:

• coding agents;

• browser agents;

• external AI review workflows;

• document management systems;

• private/public repo publication gates;

• enterprise AI governance systems;
• local-first knowledge operating systems;
• prompt routing systems;
• MCP-style tool and skill systems; and
• multi-runtime AI operations spanning desktop and cloud environments.

Candidate Claim Concepts

These are not formal claims.

1. A computer-implemented method for routing AI-agent tasks through recursive skill layers comprising core, router, and leaf skills, wherein each skill contains source pointers and machine-verifiable trigger metadata.
2. A method for preventing unsafe public or external actions by coupling skill routing with human approval boundary checks for publication, external send, credential, hook, settings, auth, and repository visibility actions.
3. A method for validating an AI operational skill layer by computing trigger gaps, required core skill presence, runtime coverage, source pointer presence, and metadata contract satisfaction.
4. A system for reducing prompt duplication by using thin runtime skills that reference source-of-truth documents rather than copying the documents into runtime prompts.
5. A publication method that separates a private full operating layer from a reduced public kernel while preserving source pointers, provenance, and reserved rights.

Public Disclosure Boundary

Before filing, do not disclose:

• complete generated skill list;
• generator source;
• private source pointers;
• internal Brain/FDE structure;
• validation script internals;
• absorbed dialogue examples;
• private runtime paths; or

• concrete external AI route registry.

Filing Checklist

• [ ] Confirm inventor names.
• [ ] Confirm owner/assignee.
• [x] Add diagrams if possible.
• [ ] Export this document to PDF.
• [x] Include generator excerpt or pseudocode if desired.
• [x] Include public/private boundary.
• [ ] File through USPTO Patent Center or chosen filing route.
• [ ] Save filing receipt and application number.
• [ ] Only after filing, consider "Patent Pending" wording.
• [ ] Calendar 12-month follow-up deadline.